

Section 83 Editorial — Implicit Treaps And Dynamic Forests

A. Sequence Cut And Paste

How to find it

Represent positions by subtree sizes and express removal/insertion through split and merge.

Correctness

Split preserves in-order order on both sides and merge concatenates its operands. The four operations therefore remove precisely $[l, r]$ and insert that unchanged block at the requested remaining position.

Complexity

$O((n+q) \log n)$ expected/amortized time and $O(n \log \sigma)$ or $O(n)$ structure memory, as appropriate.

C++

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
const ll MOD = 1000000007, BASE = 911382323;
vector<ll> pw = {1};
mt19937 rng(712367821);
struct N {
    ll v, sum, hf, hr, add = 0;
    uint32_t p = rng();
    int z = 1;
    bool rev = 0;
    N *l = 0, *r = 0;
    N(ll v) : v(v), sum(v), hf((v % MOD + MOD) % MOD), hr(hf) {}
};
int sz(N *t) { return t ? t->z : 0; }
ll sm(N *t) { return t ? t->sum : 0; }
void ensure(int n) {
    while ((int)pw.size() <= n)
        pw.push_back(pw.back() * BASE % MOD);
}
void pull(N *t) {
    if (!t)
        return;
    ensure(sz(t->l) + sz(t->r) + 1);
    t->z = 1 + sz(t->l) + sz(t->r);
    t->sum = t->v + sm(t->l) + sm(t->r);
    int l = sz(t->l), r = sz(t->r);
    ll lf = t->l ? t->l->hf : 0, rf = t->r ? t->r->hf : 0,
        lr = t->l ? t->l->hr : 0, rr = t->r ? t->r->hr : 0;
    t->hf = (lf + t->v * pw[l] + rf * pw[l + 1]) % MOD;
    t->hr = (rr + t->v * pw[r] + lr * pw[r + 1]) % MOD;
}
```

```

void add(N *t, ll x) {
    if (t)
        t->v += x, t->sum += x * t->z, t->add += x;
}

void reverse_node(N *t) {
    if (t)
        swap(t->l, t->r), swap(t->hf, t->hr), t->rev ^= 1;
}

void push(N *t) {
    if (!t)
        return;
    if (t->rev)
        reverse_node(t->l), reverse_node(t->r), t->rev = 0;
    if (t->add)
        add(t->l, t->add), add(t->r, t->add), t->add = 0;
}

pair<N *, N *> split(N *t, int k) {
    if (!t)
        return {};
    push(t);
    if (sz(t->l) >= k) {
        auto [a, b] = split(t->l, k);
        t->l = b;
        pull(t);
        return {a, t};
    }
    auto [a, b] = split(t->r, k - sz(t->l) - 1);
    t->r = a;
    pull(t);
    return {t, b};
}

N *merge(N *a, N *b) {
    if (!a || !b)
        return a ? a : b;
    if (a->p > b->p) {
        push(a);
        a->r = merge(a->r, b);
        pull(a);
        return a;
    }
    push(b);
    b->l = merge(a, b->l);
    pull(b);
    return b;
}

void output(N *t) {
    if (!t)
        return;
    push(t);
    output(t->l);
    cout << t->v << ' ';
    output(t->r);
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    N *root = 0;
    for (int i = 1; i <= n; i++)
        root = merge(root, new N(i));
    while (q--) {
        string op;

```

```

    int l, r, p;
    cin >> op >> l >> r >> p;
    auto [a, x] = split(root, l - 1);
    auto [b, c] = split(x, r - l + 1);
    root = merge(a, c);
    tie(a, c) = split(root, p - 1);
    root = merge(merge(a, b), c);
}
output(root);
cout << '\n';
}

```

Python

```

import sys

sys.setrecursionlimit(1_000_000)
MOD = 1_000_000_007
BASE = 911382323
POW = [1]
seed = 712367821

def rnd():
    global seed
    seed ^= seed << 13 & 0xFFFFFFFF
    seed ^= seed >> 17
    seed ^= seed << 5 & 0xFFFFFFFF
    return seed & 0xFFFFFFFF

class Node:
    __slots__ = "v", "p", "l", "r", "z", "sum", "add", "rev", "hf", "hr"

    def __init__(self, v):
        self.v = v
        self.p = rnd()
        self.l = self.r = None
        self.z = 1
        self.sum = v
        self.add = 0
        self.rev = False
        self.hf = self.hr = v % MOD

def size(t):
    return t.z if t else 0

def total(t):
    return t.sum if t else 0

def ensure(n):
    while len(POW) <= n:
        POW.append(POW[-1] * BASE % MOD)

def pull(t):
    if not t:
        return

```

```

ensure(size(t.l) + size(t.r) + 1)
t.z = 1 + size(t.l) + size(t.r)
t.sum = t.v + total(t.l) + total(t.r)
ll = size(t.l)
rr = size(t.r)
lf = t.l.hf if t.l else 0
rf = t.r.hf if t.r else 0
lr = t.l.hr if t.l else 0
rrh = t.r.hr if t.r else 0
t.hf = (lf + t.v * POW[ll] + rf * POW[ll + 1]) % MOD
t.hr = (rrh + t.v * POW[rr] + lr * POW[rr + 1]) % MOD

def apply_add(t, x):
    if t:
        t.v += x
        t.sum += x * t.z
        t.add += x

def apply_rev(t):
    if t:
        t.l, t.r = t.r, t.l
        t.hf, t.hr = t.hr, t.hf
        t.rev ^= True

def push(t):
    if not t:
        return
    if t.rev:
        apply_rev(t.l)
        apply_rev(t.r)
        t.rev = False
    if t.add:
        apply_add(t.l, t.add)
        apply_add(t.r, t.add)
        t.add = 0

def split(t, k):
    if not t:
        return None, None
    push(t)
    if size(t.l) >= k:
        a, t.l = split(t.l, k)
        pull(t)
        return a, t
    t.r, b = split(t.r, k - size(t.l) - 1)
    pull(t)
    return t, b

def merge(a, b):
    if not a or not b:
        return a or b
    if a.p > b.p:
        push(a)
        a.r = merge(a.r, b)
        pull(a)
        return a
    push(b)
    b.l = merge(a, b.l)

```

```

    pull(b)
    return b

def build(values):
    root = None
    for x in values:
        root = merge(root, Node(x))
    return root

def values(t, out):
    if not t:
        return
    push(t)
    values(t.l, out)
    out.append(t.v)
    values(t.r, out)

def main():
    d = sys.stdin.buffer.read().split()
    n, q = map(int, d[:2])
    root = build(range(1, n + 1))
    at = 2
    for _ in range(q):
        l, r, p = map(int, d[at + 1 : at + 4])
        at += 4
        a, b = split(root, l - 1)
        b, c = split(b, r - l + 1)
        root = merge(a, c)
        a, c = split(root, p - 1)
        root = merge(merge(a, b), c)
    out = []
    values(root, out)
    print(*out)

if __name__ == "__main__":
    main()

```

B. Reversible Range Ledger

How to find it

Attach whole-subtree addition and reversal tags to an implicit treap.

Correctness

Each tag has exactly the stated effect on the stored aggregate; pushing composes it into both children before structural inspection, and pulling restores the parent invariant.

Complexity

$O((n+q) \log n)$ expected/amortized time and $O(n \log \sigma)$ or $O(n)$ structure memory, as appropriate.

C++

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
const ll MOD = 1000000007, BASE = 911382323;
vector<ll> pw = {1};
mt19937 rng(712367821);
struct N {
    ll v, sum, hf, hr, add = 0;
    uint32_t p = rng();
    int z = 1;
    bool rev = 0;
    N *l = 0, *r = 0;
    N(ll v) : v(v), sum(v), hf((v % MOD + MOD) % MOD), hr(hf) {}
};
int sz(N *t) { return t ? t->z : 0; }
ll sm(N *t) { return t ? t->sum : 0; }
void ensure(int n) {
    while ((int)pw.size() <= n)
        pw.push_back(pw.back() * BASE % MOD);
}
void pull(N *t) {
    if (!t)
        return;
    ensure(sz(t->l) + sz(t->r) + 1);
    t->z = 1 + sz(t->l) + sz(t->r);
    t->sum = t->v + sm(t->l) + sm(t->r);
    int l = sz(t->l), r = sz(t->r);
    ll lf = t->l ? t->l->hf : 0, rf = t->r ? t->r->hf : 0,
        lr = t->l ? t->l->hr : 0, rr = t->r ? t->r->hr : 0;
    t->hf = (lf + t->v * pw[l] + rf * pw[l + 1]) % MOD;
    t->hr = (rr + t->v * pw[r] + lr * pw[r + 1]) % MOD;
}
void add(N *t, ll x) {
    if (t)
        t->v += x, t->sum += x * t->z, t->add += x;
}
void reverse_node(N *t) {
    if (t)
        swap(t->l, t->r), swap(t->hf, t->hr), t->rev ^= 1;
}
void push(N *t) {
    if (!t)
        return;
    if (t->rev)
        reverse_node(t->l), reverse_node(t->r), t->rev = 0;
    if (t->add)
        add(t->l, t->add), add(t->r, t->add), t->add = 0;
}
pair<N *, N *> split(N *t, int k) {
    if (!t)
        return {};
    push(t);
    if (sz(t->l) >= k) {
        auto [a, b] = split(t->l, k);
        t->l = b;
        pull(t);
        return {a, t};
    }
    auto [a, b] = split(t->r, k - sz(t->l) - 1);
    t->r = a;
    pull(t);
}
```

```

    return {t, b};
}
N *merge(N *a, N *b) {
    if (!a || !b)
        return a ? a : b;
    if (a->p > b->p) {
        push(a);
        a->r = merge(a->r, b);
        pull(a);
        return a;
    }
    push(b);
    b->l = merge(a, b->l);
    pull(b);
    return b;
}
void output(N *t) {
    if (!t)
        return;
    push(t);
    output(t->l);
    cout << t->v << ' ';
    output(t->r);
}
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    N *root = 0;
    for (int i = 0; i < n; i++) {
        ll x;
        cin >> x;
        root = merge(root, new N(x));
    }
    while (q--) {
        string op;
        int l, r;
        cin >> op >> l >> r;
        auto [a, x] = split(root, l - 1);
        auto [b, c] = split(x, r - l + 1);
        if (op == "ADD") {
            ll v;
            cin >> v;
            add(b, v);
        } else if (op == "REV")
            reverse_node(b);
        else
            cout << sm(b) << '\n';
        root = merge(merge(a, b), c);
    }
}

```

Python

```

import sys

sys.setrecursionlimit(1_000_000)
MOD = 1_000_000_007
BASE = 911382323
POW = [1]

```

```

seed = 712367821

def rnd():
    global seed
    seed ^= seed << 13 & 0xFFFFFFFF
    seed ^= seed >> 17
    seed ^= seed << 5 & 0xFFFFFFFF
    return seed & 0xFFFFFFFF

class Node:
    __slots__ = "v", "p", "l", "r", "z", "sum", "add", "rev", "hf", "hr"

    def __init__(self, v):
        self.v = v
        self.p = rnd()
        self.l = self.r = None
        self.z = 1
        self.sum = v
        self.add = 0
        self.rev = False
        self.hf = self.hr = v % MOD

def size(t):
    return t.z if t else 0

def total(t):
    return t.sum if t else 0

def ensure(n):
    while len(POW) <= n:
        POW.append(POW[-1] * BASE % MOD)

def pull(t):
    if not t:
        return
    ensure(size(t.l) + size(t.r) + 1)
    t.z = 1 + size(t.l) + size(t.r)
    t.sum = t.v + total(t.l) + total(t.r)
    ll = size(t.l)
    rr = size(t.r)
    lf = t.l.hf if t.l else 0
    rf = t.r.hf if t.r else 0
    lr = t.l.hr if t.l else 0
    rrrh = t.r.hr if t.r else 0
    t.hf = (lf + t.v * POW[ll] + rf * POW[ll + 1]) % MOD
    t.hr = (rrh + t.v * POW[rr] + lr * POW[rr + 1]) % MOD

def apply_add(t, x):
    if t:
        t.v += x
        t.sum += x * t.z
        t.add += x

def apply_rev(t):
    if t:

```

```

        t.l, t.r = t.r, t.l
        t.hf, t.hr = t.hr, t.hf
        t.rev ^= True

def push(t):
    if not t:
        return
    if t.rev:
        apply_rev(t.l)
        apply_rev(t.r)
        t.rev = False
    if t.add:
        apply_add(t.l, t.add)
        apply_add(t.r, t.add)
        t.add = 0

def split(t, k):
    if not t:
        return None, None
    push(t)
    if size(t.l) >= k:
        a, t.l = split(t.l, k)
        pull(t)
        return a, t
    t.r, b = split(t.r, k - size(t.l) - 1)
    pull(t)
    return t, b

def merge(a, b):
    if not a or not b:
        return a or b
    if a.p > b.p:
        push(a)
        a.r = merge(a.r, b)
        pull(a)
        return a
    push(b)
    b.l = merge(a, b.l)
    pull(b)
    return b

def build(values):
    root = None
    for x in values:
        root = merge(root, Node(x))
    return root

def values(t, out):
    if not t:
        return
    push(t)
    values(t.l, out)
    out.append(t.v)
    values(t.r, out)

def main():
    d = sys.stdin.buffer.read().split()

```

```

n, q = map(int, d[:2])
root = build(map(int, d[2 : 2 + n]))
at = 2 + n
out = []
for _ in range(q):
    op = d[at]
    l = int(d[at + 1]) - 1
    r = int(d[at + 2])
    at += 3
    a, b = split(root, l)
    b, c = split(b, r - 1)
    if op == b"ADD":
        apply_add(b, int(d[at]))
        at += 1
    elif op == b"REV":
        apply_rev(b)
    else:
        out.append(str(total(b)))
    root = merge(merge(a, b), c)
print("\n".join(out))

if __name__ == "__main__":
    main()

```

C. Dynamic Forest XOR

How to find it

Expose each represented path with `makeroot(u)`; `access(v)` and aggregate its auxiliary tree.

Correctness

After exposure, the auxiliary subtree rooted at `v` contains exactly the `u-v` path in order. XOR is associative and reversal-independent, so its stored aggregate is the answer.

Complexity

$O((n+q) \log n)$ expected/amortized time and $O(n \log \sigma)$ or $O(n)$ structure memory, as appropriate.

C++

```

#include <bits/stdc++.h>
using namespace std;
using ll = long long;
struct LCT {
    int n, mode;
    ll mod;
    vector<array<int, 2>> ch;
    vector<int> p, rev, sz;
    vector<ll> val, agg, mul, add;
    LCT(vector<ll> a, int mode = 0, ll mod = 0)
        : n(a.size()), mode(mode), mod(mod), ch(n + 1), p(n + 1), rev(n + 1),
          sz(n + 1, 1), val(n + 1), agg(n + 1), mul(n + 1, 1), add(n + 1) {}
    sz[0] = 0;
    for (int i = 1; i <= n; i++)
        val[i] = agg[i] = a[i - 1];
}

```

```

bool root(int x) { return !p[x] || (ch[p[x]][0] != x && ch[p[x]][1] != x); }
void pull(int x) {
    auto [l, r] = ch[x];
    sz[x] = 1 + sz[l] + sz[r];
    agg[x] = mode ? (agg[l] ^ val[x] ^ agg[r]) : agg[l] + val[x] + agg[r];
    if (mod)
        agg[x] %= mod;
}
void reverse_node(int x) {
    if (x)
        swap(ch[x][0], ch[x][1]), rev[x] ^= 1;
}
void affine(int x, ll a, ll b) {
    if (!x)
        return;
    val[x] = (a * val[x] + b) % mod;
    agg[x] = (a * agg[x] + b * sz[x]) % mod;
    mul[x] = a * mul[x] % mod;
    add[x] = (a * add[x] + b) % mod;
}
void push(int x) {
    if (rev[x])
        reverse_node(ch[x][0]), reverse_node(ch[x][1]), rev[x] = 0;
    if (mul[x] != 1 || add[x]) {
        affine(ch[x][0], mul[x], add[x]);
        affine(ch[x][1], mul[x], add[x]);
        mul[x] = 1;
        add[x] = 0;
    }
}
void rotate(int x) {
    int y = p[x], z = p[y], s = ch[y][1] == x, b = ch[x][s ^ 1];
    if (!root(y))
        ch[z][ch[z][1] == y] = x;
    p[x] = z;
    ch[x][s ^ 1] = y;
    p[y] = x;
    ch[y][s] = b;
    if (b)
        p[b] = y;
    pull(y);
    pull(x);
}
void splay(int x) {
    vector<int> st = {x};
    for (int y = x; !root(y);)
        y = p[y], st.push_back(y);
    while (!st.empty())
        push(st.back(), st.pop_back());
    while (!root(x)) {
        int y = p[x], z = p[y];
        if (!root(y))
            rotate((ch[y][1] == x) == (ch[z][1] == y) ? y : x);
        rotate(x);
    }
}
void access(int x) {
    for (int y = 0, z = x; z; y = z, z = p[z])
        splay(z), ch[z][1] = y, pull(z);
    splay(x);
}
void makeroot(int x) {
    access(x);
}

```

```

    reverse_node(x);
}
void link(int x, int y) {
    makeroot(x);
    p[x] = y;
}
void cut(int x, int y) {
    makeroot(x);
    access(y);
    ch[y][0] = 0;
    p[x] = 0;
    pull(y);
}
int path(int x, int y) {
    makeroot(x);
    access(y);
    return y;
}
void setv(int x, ll v) {
    access(x);
    val[x] = v;
    pull(x);
}
};
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<ll> a(n);
    for (auto &x : a)
        cin >> x;
    LCT t(a, 1, 0);
    while (q--) {
        string op;
        cin >> op;
        if (op == "LINK") {
            int u, v;
            cin >> u >> v;
            t.link(u, v);
        } else if (op == "CUT") {
            int u, v;
            cin >> u >> v;
            t.cut(u, v);
        } else if (op == "SET") {
            int u;
            ll x;
            cin >> u >> x;
            t.setv(u, x);
        } else if (op == "AFFINE") {
            int u, v;
            ll x, y;
            cin >> u >> v >> x >> y;
            int z = t.path(u, v);
            t.affine(z, x, y);
        } else {
            int u, v;
            cin >> u >> v;
            cout << t.agg[t.path(u, v)] << '\n';
        }
    }
}
}

```

Python

```
import sys

class LinkCut:
    def __init__(self, values, mode="sum", mod=None):
        n = len(values)
        self.n = n
        self.ch = [[0, 0] for _ in range(n + 1)]
        self.p = [0] * (n + 1)
        self.rev = [0] * (n + 1)
        self.val = [0] + values[:]
        self.agg = [0] + values[:]
        self.sz = [0] + [1] * n
        self.mode = mode
        self.mod = mod
        self.mul = [1] * (n + 1)
        self.add = [0] * (n + 1)

    def root(self, x):
        p = self.p[x]
        return not p or (self.ch[p][0] != x and self.ch[p][1] != x)

    def pull(self, x):
        l, r = self.ch[x]
        self.sz[x] = 1 + self.sz[l] + self.sz[r]
        self.agg[x] = (
            (self.agg[l] ^ self.val[x] ^ self.agg[r])
            if self.mode == "xor"
            else self.agg[l] + self.val[x] + self.agg[r]
        )
        if self.mod:
            self.agg[x] %= self.mod

    def reverse(self, x):
        if x:
            self.ch[x].reverse()
            self.rev[x] ^= 1

    def affine(self, x, a, b):
        if not x:
            return
        mod = self.mod
        self.val[x] = (a * self.val[x] + b) % mod
        self.agg[x] = (a * self.agg[x] + b * self.sz[x]) % mod
        self.mul[x] = a * self.mul[x] % mod
        self.add[x] = (a * self.add[x] + b) % mod

    def push(self, x):
        if self.rev[x]:
            self.reverse(self.ch[x][0])
            self.reverse(self.ch[x][1])
            self.rev[x] = 0
        if self.mul[x] != 1 or self.add[x]:
            a, b = self.mul[x], self.add[x]
            self.affine(self.ch[x][0], a, b)
            self.affine(self.ch[x][1], a, b)
            self.mul[x] = 1
            self.add[x] = 0

    def rotate(self, x):
        p = self.p[x]
```

```

    g = self.p[p]
    side = self.ch[p][1] == x
    b = self.ch[x][side ^ 1]
    if not self.root(p):
        self.ch[g][self.ch[g][1] == p] = x
    self.p[x] = g
    self.ch[x][side ^ 1] = p
    self.p[p] = x
    self.ch[p][side] = b
    if b:
        self.p[b] = p
    self.pull(p)
    self.pull(x)

def splay(self, x):
    path = [x]
    y = x
    while not self.root(y):
        y = self.p[y]
        path.append(y)
    for y in reversed(path):
        self.push(y)
    while not self.root(x):
        p = self.p[x]
        g = self.p[p]
        if not self.root(p):
            self.rotate(
                p if (self.ch[p][1] == x) == (self.ch[g][1] == p) else x
            )
        self.rotate(x)

def access(self, x):
    last = 0
    y = x
    while y:
        self.splay(y)
        self.ch[y][1] = last
        self.pull(y)
        last = y
        y = self.p[y]
    self.splay(x)

def makeroot(self, x):
    self.access(x)
    self.reverse(x)

def findroot(self, x):
    self.access(x)
    while self.push(x) is None and self.ch[x][0]:
        x = self.ch[x][0]
    self.splay(x)
    return x

def link(self, u, v):
    self.makeroot(u)
    self.p[u] = v

def cut(self, u, v):
    self.makeroot(u)
    self.access(v)
    self.ch[v][0] = 0
    self.p[u] = 0
    self.pull(v)

```

```

def path(self, u, v):
    self.makeroot(u)
    self.access(v)
    return v

def set(self, u, x):
    self.access(u)
    self.val[u] = x
    self.pull(u)

def main():
    d = sys.stdin.buffer.read().split()
    n, q = map(int, d[:2])
    t = LinkCut(list(map(int, d[2 : 2 + n])), "xor", None)
    at = 2 + n
    out = []
    for _ in range(q):
        op = d[at]
        at += 1
        if op == b"LINK":
            u, v = map(int, d[at : at + 2])
            at += 2
            t.link(u, v)
        elif op == b"CUT":
            u, v = map(int, d[at : at + 2])
            at += 2
            t.cut(u, v)
        elif op == b"SET":
            u, x = map(int, d[at : at + 2])
            at += 2
            t.set(u, x)
        elif op == b"AFFINE":
            u, v, a, b = map(int, d[at : at + 4])
            at += 4
            x = t.path(u, v)
            t.affine(x, a, b)
        else:
            u, v = map(int, d[at : at + 2])
            at += 2
            out.append(str(t.agg[t.path(u, v)]))
    print("\n".join(out))

if __name__ == "__main__":
    main()

```

D. Threshold Component Size

How to find it

Kruskal creates a new parent whenever two components merge; label it with the edge weight and component size.

Correctness

Ancestors of a leaf are exactly its successive Kruskal components in nondecreasing threshold order. The highest ancestor of weight at most x is therefore precisely the requested component.

Complexity

$O((n+q) \log n)$ expected/amortized time and $O(n \log \sigma)$ or $O(n)$ structure memory, as appropriate.

C++

```
#include <bits/stdc++.h>
using namespace std;
struct KRT {
    int n, z, L;
    vector<int> dsu, parent, dep, weight, sz, component;
    vector<vector<int>>> up, child;
    int find(int x) { return dsu[x] == x ? x : dsu[x] = find(dsu[x]); }
    KRT(int n, vector<array<int, 3>> e)
        : n(n), z(z), dsu(2 * n + 5), parent(2 * n + 5), dep(2 * n + 5),
          weight(2 * n + 5, -1), sz(2 * n + 5, 1), child(2 * n + 5) {
        iota(dsu.begin(), dsu.end(), 0);
        iota(parent.begin(), parent.end(), 0);
        sort(e.begin(), e.end());
        for (auto [w, u, v] : e) {
            u = find(u);
            v = find(v);
            if (u == v)
                continue;
            int x = z++;
            weight[x] = w;
            sz[x] = sz[u] + sz[v];
            child[x] = {u, v};
            parent[u] = parent[v] = x;
            dsu[u] = dsu[v] = dsu[x] = x;
        }
        vector<int> roots;
        for (int i = 0; i < z; i++)
            if (parent[i] == i)
                roots.push_back(i);
        for (int r : roots) {
            vector<int> st = {r};
            while (!st.empty()) {
                int x = st.back();
                st.pop_back();
                for (int y : child[x])
                    dep[y] = dep[x] + 1, st.push_back(y);
            }
        }
        L = max(1, (int)bit_width((unsigned)z));
        up.assign(L, vector<int>(z));
        for (int i = 0; i < z; i++)
            up[0][i] = parent[i];
        for (int j = 1; j < L; j++)
            for (int i = 0; i < z; i++)
                up[j][i] = up[j - 1][up[j - 1][i]];
        component.resize(n);
        for (int i = 0; i < n; i++)
            component[i] = find(i);
    }
    int climb(int v, int x) {
        for (int j = L - 1; j >= 0; j--) {
            int q = up[j][v];
            if (q != v && weight[q] <= x)
                v = q;
        }
        return v;
    }
};
```

```

}
int lca(int a, int b) {
    if (component[a] != component[b])
        return -1;
    if (dep[a] < dep[b])
        swap(a, b);
    int d = dep[a] - dep[b];
    for (int j = 0; j < L; j++)
        if (d >> j & 1)
            a = up[j][a];
    if (a == b)
        return a;
    for (int j = L - 1; j >= 0; j--)
        if (up[j][a] != up[j][b])
            a = up[j][a], b = up[j][b];
    return up[0][a];
}
};
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, q;
    cin >> n >> m >> q;
    vector<array<int, 3>> e(m);
    for (auto &[w, u, v] : e)
        cin >> u >> v >> w, --u, --v;
    KRT t(n, e);
    while (q--) {
        int v, x;
        cin >> v >> x;
        cout << t.sz[t.climb(v - 1, x)] << '\n';
    }
}

```

Python

```

import sys

class KRT:
    def __init__(self, n, edges):
        z = 2 * n + 5
        self.parent = list(range(z))
        self.tree = [[] for _ in range(z)]
        self.weight = [-1] * z
        self.size = [1] * z
        self.next = n
        dsu = list(range(z))

        def find(x):
            while dsu[x] != x:
                dsu[x] = dsu[dsu[x]]
                x = dsu[x]
            return x

        for w, u, v in sorted(edges):
            a, b = find(u), find(v)
            if a == b:
                continue
            x = self.next
            self.next += 1

```

```

        self.weight[x] = w
        self.size[x] = self.size[a] + self.size[b]
        self.tree[x] = [a, b]
        self.parent[a] = self.parent[b] = x
        dsu[a] = dsu[b] = dsu[x] = x
    roots = [v for v in range(self.next) if self.parent[v] == v]
    self.depth = [0] * self.next
    order = []
    for root in roots:
        st = [root]
        while st:
            x = st.pop()
            order.append(x)
            for y in self.tree[x]:
                self.depth[y] = self.depth[x] + 1
                st.append(y)
    self.L = max(1, self.next.bit_length())
    self.up = [[0] * self.next for _ in range(self.L)]
    self.up[0] = self.parent[: self.next]
    for j in range(1, self.L):
        self.up[j] = [
            self.up[j - 1][self.up[j - 1][x]] for x in range(self.next)
        ]
    self.component = [self.up[-1][v] for v in range(n)]

def climb(self, v, x):
    for j in range(self.L - 1, -1, -1):
        p = self.up[j][v]
        if p != v and self.weight[p] <= x:
            v = p
    return v

def lca(self, a, b):
    if self.component[a] != self.component[b]:
        return -1
    if self.depth[a] < self.depth[b]:
        a, b = b, a
    d = self.depth[a] - self.depth[b]
    for j in range(self.L):
        if d >> j & 1:
            a = self.up[j][a]
    if a == b:
        return a
    for j in range(self.L - 1, -1, -1):
        if self.up[j][a] != self.up[j][b]:
            a, b = self.up[j][a], self.up[j][b]
    return self.up[0][a]

def main():
    d = list(map(int, sys.stdin.buffer.read().split()))
    n, m, q = d[:3]
    at = 3
    e = []
    for i in range(m):
        u, v, w = d[at : at + 3]
        at += 3
        e.append((w, u - 1, v - 1))
    t = KRT(n, e)
    print(
        "\n".join(
            str(t.size[t.climb(d[i] - 1, d[i + 1])])
            for i in range(at, at + 2 * q, 2)
        )
    )

```

```
    )  
    )  
  
if __name__ == "__main__":  
    main()
```

Upsolving and stress testing

For A/B, maintain a literal Python-style list and apply every cut, paste, reverse, and addition directly. For C, maintain the represented forest as adjacency sets and reconstruct each path with BFS. For D, replay all edges up to the queried threshold and run DSU from scratch.

Adversarial sequences should repeatedly reverse overlapping intervals, compose updates before any query forces a push, link and cut the same edge, query a single vertex, and use equal reconstruction thresholds. The central invariant is always which order or tree is represented; randomized storage and preferred paths are implementation details.